

THE DEV^{OPS} ENGINEER'S DAILY OPERATIONS HANDBOOK

How Production DevOps Engineers
Run, Monitor, Troubleshoot,
and Protect Systems Every Day



MONITOR



OPERATE



TROUBLESHOOT



PROTECT



AUTOMATE



PRACTICAL. REPEATABLE. RELIABLE.
BUILT FOR REAL-WORLD OPERATIONS.

1. STARTING THE DAY: PRODUCTION HEALTH CHECK

Understand what an experienced DevOps engineer checks before touching production.

- 1  Overnight incidents and alerts
- 2  Current service health
- 3  Deployment activity
- 4  Failed CI/CD pipelines
- 5  Kubernetes cluster health
- 6  Infrastructure changes
- 7  Cloud provider health
- 8  Backup status
- 9  Certificate expiration
- 10  Capacity warnings
- 11  Scheduled maintenance
- 12  Open incidents
- 13  Handover notes

BUILDING A 10-MINUTE MORNING HEALTH CHECK

- | | | |
|----------|--|---|
| 0-2 MIN |  | Scan alerts, incidents and service health overview |
| 2-4 MIN |  | Review deployments and failed pipelines |
| 4-6 MIN |  | Check Kubernetes cluster and node health |
| 6-8 MIN |  | Validate cloud infrastructure and capacity |
| 8-10 MIN |  | Confirm backups, certificates, maintenance and open incidents |



Goal: Understand the state of production before acting.



WHAT DESERVES IMMEDIATE ATTENTION

Active incidents, service down, security alerts, capacity critical, failed deployments.



WHAT CAN SAFELY WAIT

Low-priority alerts, feature requests, non-urgent changes, improvements not affecting availability.



SENIOR ENGINEER TIP

Never start changing production before understanding its current state.



2. READING ALERTS WITHOUT CHASING NOISE

Turn monitoring alerts into operational decisions.

- 1  Alert vs incident
- 2  Symptom vs root cause
- 3  Warning vs critical alerts
- 4  Actionable alerts
- 5  Alert fatigue
- 6  Duplicate alerts
- 7  Cascading alerts
- 8  Identifying the first failure
- 9  Correlating timestamps
- 10  Checking recent changes
- 11  Infrastructure vs application symptoms
- 12  SLI and SLO context
- 13  When not to react
- 14  Escalation thresholds
- 15  Silencing alerts safely



PRODUCTION NOTE

The loudest alert is not always the root cause.



ALERT TO ACTION FRAMEWORK

- 1  **Observe**
What is happening right now?
- 2  **Understand**
What is the real problem?
- 3  **Prioritize**
What needs immediate action?
- 4  **Act**
Take the right operational step.
- 5  **Verify**
Confirm the system is healthy again.

GOOD ALERTS ARE:

- ✓ Specific
- ✓ Actionable
- ✓ Owned
- ✓ Measurable
- ✓ Relevant
- ✓ Timely



BAD ALERTS ARE:

- ✗ Vague
- ✗ Noisy
- ✗ Unowned
- ✗ Duplicated
- ✗ Lacking context
- ✗ Always firing

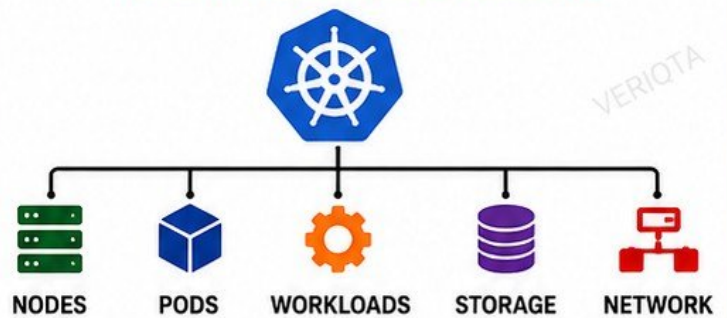


3. CHECKING KUBERNETES BEFORE PROBLEMS BECOME INCIDENTS

Perform a practical daily Kubernetes health review.

- 1  Node readiness
- 2  Pod health
- 3  Restart counts
- 4  Pending Pods
- 5  CrashLoopBackOff
- 6  OOMKilled workloads
- 7  Failed Jobs
- 8  Deployment availability
- 9  DaemonSet coverage
- 10  StatefulSet health
- 11  CPU and memory pressure
- 12  PersistentVolume status
- 13  Kubernetes events
- 14  DNS health
- 15  Ingress health
- 16  Certificate status

KUBERNETES HEALTH OVERVIEW



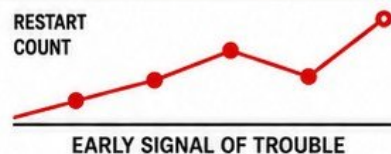
USEFUL kubectl COMMANDS

<code>kubectl get nodes</code>	Check node status
<code>kubectl get pods -A</code>	List all Pods in all namespaces
<code>kubectl get pods -A -o wide</code>	Pods with node and IP info
<code>kubectl get pods -A --field-selector status.phase=Pending</code>	Find Pending Pods
<code>kubectl get pods -A --sort-by=.status.containerStatuses[0].restartCount</code>	List Pods by restart count
<code>kubectl get events -A --sort-by=.lastTimestamp</code>	See recent events
<code>kubectl top nodes</code>	Node CPU and memory usage
<code>kubectl top pods -A</code>	Pod CPU and memory usage
<code>kubectl get pvc -A</code>	Check PersistentVolume status
<code>kubectl get ing -A</code>	Check Ingress resources
<code>kubectl get certificates -A</code>	Check certificate objects



DEBUGGING INSIGHT

Restart counts often expose problems before alerts fire.



4. WATCHING CPU, MEMORY, DISK, AND NETWORK TRENDS

Know when infrastructure is approaching failure.

- 1  CPU utilization vs saturation
- 2  Load average
- 3  Memory utilization
- 4  Available memory
- 5  Swap activity
- 6  OOM events
- 7  Disk utilization
- 8  Inode exhaustion
- 9  Disk I/O latency
- 10  Network throughput
- 11  Packet drops
- 12  Connection counts
- 13  Capacity trends
- 14  Baselines
- 15  Headroom
- 16  Why 80% utilization does not always mean danger

KEY METRICS AT A GLANCE

CPU



MEMORY



DISK



NETWORK



LOAD AVG



1.24
(1m)

INODES



HEALTH ZONE GUIDE

- 0 - 60% **Healthy**
(Plenty of headroom)
- 60 - 80% **Watch**
(Close monitoring required)
- 80 - 90% **At Risk**
(Take action soon)
- 90%+ **Critical**
(High chance of failure)

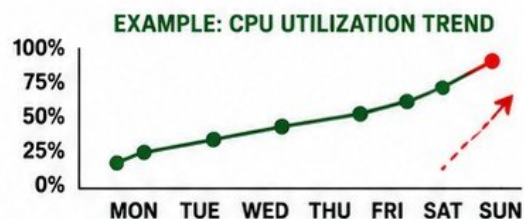
WHY 80% IS NOT ALWAYS DANGEROUS

- ✓ Burstable workloads
- ✓ Short spikes
- ✓ Good caching
- ✓ High idle capacity elsewhere
- ✓ Strong redundancy
- ✓ Healthy response times



PERFORMANCE TIP

Trend direction matters more than one isolated metric.



Rising trend is a warning sign.

5. REVIEWING DEPLOYMENTS AND PRODUCTION CHANGES

Know exactly what changed before troubleshooting.

- 1  Recent application deployments
- 2  Infrastructure changes
- 3  Configuration changes
- 4  Feature flags
- 5  Secret rotations
- 6  Dependency upgrades
- 7  Database migrations
- 8  Kubernetes rollout history
- 9  Git commit correlation
- 10  CI/CD execution history
- 11  Change windows
- 12  Deployment annotations
- 13  Comparing before and after
- 14  Rollback readiness
- 15  Change ownership

CHANGE INVESTIGATION FLOW

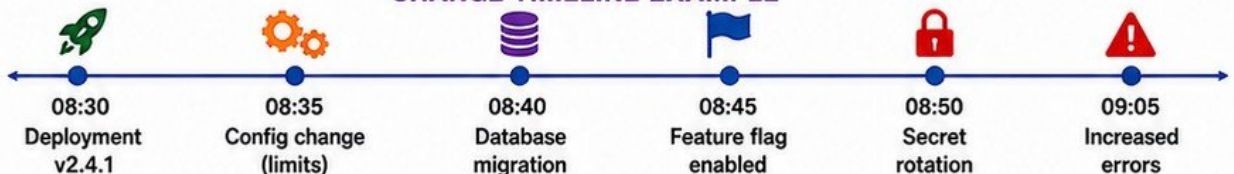


- | | | | |
|-------------------------------------|---|--|--|
| 1. IDENTIFY
What changed? | 2. VERIFY
When did it happen? | 3. CORRELATE
Is it related to the issue? | 4. DECIDE
Continue, rollback, or mitigate? |
|-------------------------------------|---|--|--|

WHAT TO CHECK

-  Timeline of recent changes
-  Deployment start and end times
-  Who made the change?
-  What exactly was changed?
-  Commit message and reference
-  Impact on services and dependencies
-  Risk level of the change

CHANGE TIMELINE EXAMPLE



REAL WORLD LESSON

Always ask "what changed?"
before assuming infrastructure failed.



6. OPERATING CI/CD PIPELINES EVERY DAY

Keep delivery systems reliable instead of treating pipelines as background automation.

- 1  Failed pipeline review
- 2  Flaky pipelines
- 3  Queue time
- 4  Runner and agent capacity
- 5  Build duration changes
- 6  Dependency failures
- 7  Artifact failures
- 8  Registry availability
- 9  Authentication failures
- 10  Expired credentials
- 11  Deployment failures
- 12  Stuck jobs
- 13  Retry vs investigate
- 14  Pipeline observability
- 15  Pipeline ownership
- 16  Preventing repeated failures

CI/CD PIPELINE HEALTH CHECK

			
Are pipelines failing?	Are failures random or patterned?	Are queues growing?	Are runners saturated?
			
Are builds taking longer?	Are artifacts and registry healthy?	Are credentials valid and rotating?	Are deployments succeeding consistently?

RESPONSE GUIDE

	INVESTIGATE Find the root cause, check logs, and recent changes.
	FIX Address the issue in code, config, or infrastructure.
	VERIFY Confirm the fix by running the pipeline successfully.
	IMPROVE Document the cause and prevent the issue from repeating.



PRODUCTION NOTE

A pipeline that requires constant manual retries is already unhealthy.



CODE



BUILD



TEST



DEPLOY



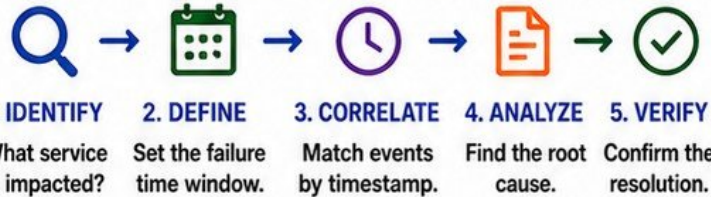
SUCCESS

7. READING LOGS LIKE A PRODUCTION ENGINEER

Move from millions of log lines to the useful evidence.

- 1  Start with the affected service
- 2  Define the failure window
- 3  Correlate timestamps
- 4  Request IDs
- 5  Trace IDs
- 6  Error patterns
- 7  Status codes
- 8  Application logs
- 9  Container logs
- 10  Kubernetes logs
- 11  System logs
- 12  Proxy and load balancer logs
- 13  Cloud logs
- 14  Filtering noise
- 15  Comparing healthy vs failed requests
- 16  Multi-service correlation

LOG INVESTIGATION FLOW



LOG SOURCES TO CHECK

	APPLICATION LOGS Business logic, errors, and exceptions.
	CONTAINER LOGS Stdout, stderr, and application output.
	KUBERNETES LOGS Pod events, scheduling, and cluster events.
	SYSTEM LOGS OS-level logs (auth, kernel, services).
	PROXY & LOAD BALANCER LOGS Request routing, status codes, latency.
	CLOUD LOGS Cloud services, APIs, IAM, and audit logs.

USEFUL LOG CONTEXT

					
TIMESTAMP When did it happen?	REQUEST ID Which request was affected?	TRACE ID How did the request flow across services?	ERROR What went wrong?	STATUS CODE How did the system respond?	USER / CLIENT Who or what made the request?



DEBUGGING INSIGHT

Search around the failure, not across everything.



VERIQTA
YOUTUBE



VERIQTA
GITHUB



VERIQTA
LINKEDIN



VERIQTA
X



VERIQTA
INSTAGRAM



VERIQTA
TELEGRAM

8. HANDLING DISK SPACE BEFORE PRODUCTION STOPS

Treat storage exhaustion as an operational risk.

- 1  df vs du
- 2  Inodes
- 3  Large directories
- 4  Growing logs
- 5  Container storage
- 6  Docker image accumulation
- 7  Kubernetes ephemeral storage
- 8  Temporary files
- 9  Deleted but open files
- 10  Log rotation
- 11  Database storage
- 12  Persistent volumes
- 13  Safe cleanup
- 14  Dangerous cleanup commands
- 15  Capacity forecasting
- 16  Disk alerts

KEY DISK METRICS TO MONITOR

DISK USAGE



Used
78%

INODES



Used
82%

INODE USAGE



Used
68%

SPACE LEFT



Free
22 GB

USEFUL COMMANDS

```
df -h
df -i
du -sh *
du -h --max-depth=1 /path
du -sh /var/log/*
lsof +L1
docker system df
docker image prune -a
kubectl top node
find /tmp -type f -mtime +7 -delete
```

```
Show disk usage
Show inode usage
Top-level directory sizes
Largest directories
Check log directory sizes
Find deleted but open files
Docker disk usage summary
Remove unused Docker images
Check node storage usage
Clean old temp files (safe)
```



SAFE CLEANUP EXAMPLES

- Remove old logs after rotation
- Clean expired cache files
- Prune unused Docker images
- Remove old temporary files
- Archive old application logs



DANGEROUS COMMANDS (AVOID)

- `rm -rf /`
- `rm -rf /var/log/*` (without rotation)
- Truncating active log files
- Deleting unknown directories
- Removing files without backup

DISK SPACE CHECK FLOW



1. CHECK
Disk usage
(df -h)



2. CHECK
Inodes
(df -i)



3. FIND
Large directories
(du -sh *)



4. IDENTIFY
What is growing
and why



5. CLEAN
Safely remove
unnecessary data



6. MONITOR
Set alerts and
forecast usage



PRODUCTION FAILURE

100% disk usage can break services that appear unrelated to storage.



DATABASE
FAILURES



WEB APP
DOWNTIME



CONTAINER
CRASHES



MONITORING
FAILURES



LOAD BALANCER
FAILURES



VERIQTA
YOUTUBE



VERIQTA
GITHUB



VERIQTA
LINKEDIN



VERIQTA
X



VERIQTA
INSTAGRAM



VERIQTA
TELEGRAM

9. MANAGING SECRETS, CERTIFICATES, AND EXPIRATION RISK

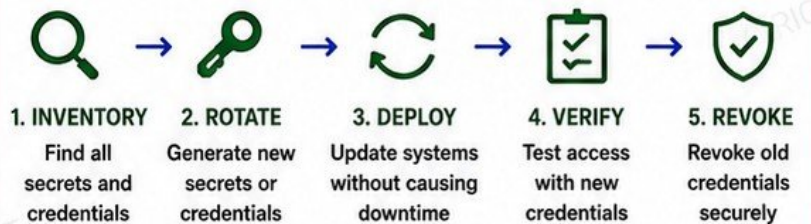
Prevent predictable outages caused by expiring credentials.

- 1  TLS certificates
- 2  API tokens
- 3  Cloud credentials
- 4  Kubernetes Secrets
- 5  CI/CD credentials
- 6  Service account credentials
- 7  SSH keys
- 8  Certificate expiration monitoring
- 9  Secret rotation
- 10  Rotation without downtime
- 11  Ownership
- 12  Automated renewal
- 13  Testing renewed credentials
- 14  Revocation
- 15  Audit trails

EXPIRATION RISK AT A GLANCE

TLS CERTIFICATES	API TOKENS	CLOUD CREDENTIALS	K8S SECRETS
			
7 days expiring	14 days expiring	3 days expiring	OK > 30 days

SECRET ROTATION LIFECYCLE



EXPIRATION MONITORING EXAMPLE

RESOURCE	EXPIRES IN	STATUS	ACTION
TLS Certificate	7 days	CRITICAL	Renew now
API Token	14 days	WARNING	Rotate soon
Cloud IAM Key	3 days	CRITICAL	Rotate now
K8s Secret	30+ days	HEALTHY	Monitor
Service Account	45 days	WARNING	Rotate soon

WHERE SECRETS LIVE

 KUBERNETES SECRETS	 AWS SECRETS MANAGER	 GCP SECRET MANAGER	 AZURE KEY VAULT	 CI/CD SECURE STORAGE	 APPLICATION CONFIG
--	---	--	---	--	--



SECURITY REMINDER

Every production secret needs an owner and lifecycle.

 **OWNER**
Someone is accountable

 **LIFECYCLE**
Rotate, renew, and revoke

 **MONITORING**
Track expirations continuously

 **DOCUMENTATION**
Know what exists and where

 **REVIEW**
Review access regularly

10. DAILY AWS AND CLOUD INFRASTRUCTURE OPERATIONS

Watch the cloud infrastructure behind the application.

- 1  EC2 health
- 2  Auto Scaling Groups
- 3  Load balancer target health
- 4  RDS health
- 5  EBS capacity
- 6  S3 operational signals
- 7  VPC and networking issues
- 8  NAT Gateway behavior
- 9  Route changes
- 10  IAM failures
- 11  CloudWatch alarms
- 12  CloudTrail events
- 13  Resource quotas
- 14  Service limits
- 15  Unexpected resource creation
- 16  Cloud provider incidents
- 17  Cost anomalies
- 18  Security group changes
- 19  KMS key status

DAILY CLOUD HEALTH DASHBOARD

EC2 INSTANCES



Healthy

AUTO SCALING



In Service

LOAD BALANCERS



All Targets OK

RDS DATABASES



Healthy

EBS VOLUMES



OK

S3 BUCKETS



Operational

VPC NETWORK



No Issues

CLOUDWATCH ALARMS



2 Active

KEY DAILY CHECKS

- ✓ Verify instance status and reachability
- ✓ Confirm Auto Scaling capacity
- ✓ Check load balancer target health
- ✓ Review database connections and storage
- ✓ Validate backup completion
- ✓ Monitor networking and routes
- ✓ Review CloudWatch alarms
- ✓ Check IAM and permission errors
- ✓ Inspect CloudTrail for unusual activity
- ✓ Review service limits and quotas
- ✓ Look for unexpected resources
- ✓ Validate security groups and NACLs
- ✓ Monitor NAT Gateway metrics
- ✓ Confirm DNS and Route 53 health
- ✓ Track cost and usage anomalies
- ✓ Check provider health dashboard
- ✓ Review open support cases
- ✓ Document findings and actions

CLOUD OPERATIONS FLOW



1. OBSERVE
Collect metrics and signals

2. ANALYZE
Identify anomalies and trends

3. VALIDATE
Confirm impact and scope

4. RESPOND
Mitigate or escalate

5. DOCUMENT
Record actions and learnings

6. IMPROVE
Prevent repeated issues



ENTERPRISE ADVICE

Cloud resources should be monitored as dependencies, not isolated components.



APPLICATION



INFRASTRUCTURE



SECURITY



COST



RELIABILITY



VERIQT
YOUTUBE



VERIQT
GITHUB



VERIQT
LINKEDIN



VERIQT
X



VERIQT
INSTAGRAM



VERIQT
TELEGRAM

11. INFRASTRUCTURE AS CODE OPERATIONAL DISCIPLINE

Keep Terraform and infrastructure changes predictable.

- 1  Repository state
- 2  Terraform plan review
- 3  Unexpected drift
- 4  State locking
- 5  Remote state
- 6  Failed applies
- 7  Partial changes
- 8  Provider changes
- 9  Module version changes
- 10  Resource replacement
- 11  Destructive plans
- 12  Importing unmanaged resources
- 13  Manual console changes
- 14  Production approvals
- 15  Plan artifacts
- 16  State backup

TERRAFORM LIFECYCLE OVERVIEW



PLAN REVIEW CHECKLIST

- | | |
|---|--|
| ☒ Review all changes carefully | ☒ Confirm environment targets |
| ☒ Check for resource replacements | ☒ Review cost impact |
| ☒ Look for destructive operations | ☒ Validate module version updates |
| ☒ Validate provider and version changes | ☒ Ensure backups are in place |
| ☒ Confirm no unexpected drift | ☒ Document the reason for change |
| ☒ Ensure dependencies are correct | ☒ Attach plan artifact to approval |
| ☒ Verify tagging and naming standards | ☒ Obtain required approvals |
| ☒ Check IAM and permission impact | ☒ Schedule maintenance window |

COMMON RISKS TO WATCH

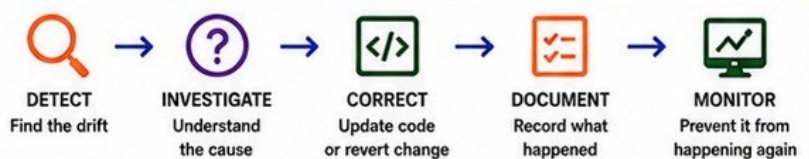


STATE MANAGEMENT BEST PRACTICES



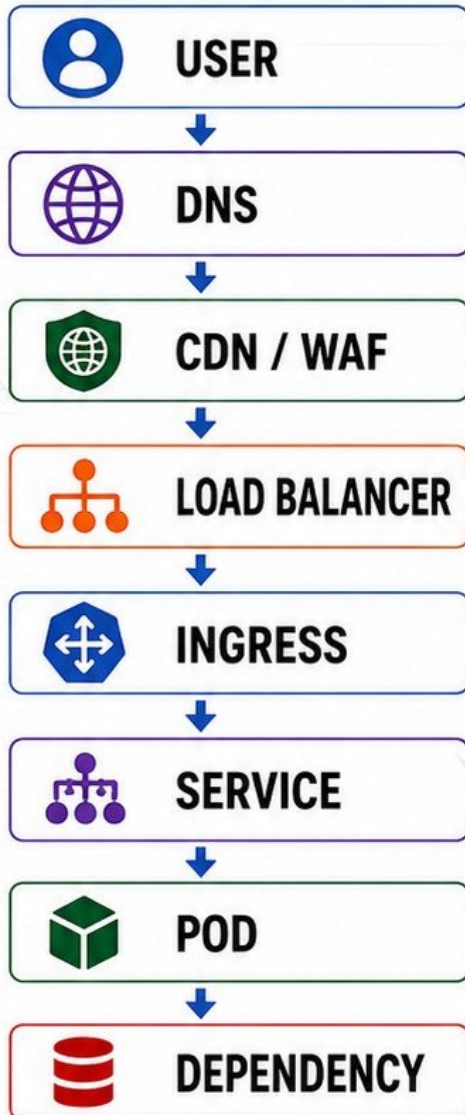
SENIOR ENGINEER TIP

Never normalize unexplained Terraform drift.



12. DNS, LOAD BALANCERS, AND TRAFFIC FLOW CHECKS

Troubleshoot the path between users and applications.



- 1 DNS resolution
- 2 TTL behavior
- 3 Certificate validation
- 4 Load balancer health
- 5 Target health
- 6 Listener rules
- 7 Security groups
- 8 Kubernetes Ingress
- 9 Service endpoints
- 10 Pod readiness
- 11 Upstream timeouts
- 12 502 vs 503 vs 504
- 13 Network path isolation



DEBUGGING INSIGHT

Follow the request hop by hop instead of guessing.

PRO TIP

Start from the user side and move downstream. Verify each hop before moving on.

DATABASE AND DEPENDENCY HEALTH FROM THE DEVOPS SIDE



RECOGNIZE INFRASTRUCTURE SYMPTOMS CAUSED BY DOWNSTREAM SYSTEMS.

1. DATABASE HEALTH

- **CONNECTION POOLS**
→ Too many open connections increase latency and errors.
- **CONNECTION EXHAUSTION**
→ No free connections = timeouts.
- **DATABASE LATENCY**
→ High query time affects app response.
- **CPU AND MEMORY**
→ High usage slows queries and increases timeouts.
- **STORAGE**
→ Slow disks or full storage cause DB stalls.
- **REPLICATION LAG**
→ Stale reads, failover delays, data inconsistency.
- **QUERY LATENCY**
→ Slow queries hold locks and block other requests.



2. CACHE HEALTH (REDIS)

- **REDIS MEMORY**
→ High memory usage → evictions, OOM, slow responses.
- **HIT RATE**
→ Low hit rate = unnecessary DB load.
- **LATENCY**
→ High latency impacts app speed.
- **CONNECTIONS**
→ Too many clients → instability.



3. QUEUE HEALTH

- **QUEUE DEPTH**
→ Growing queue = processing slower than incoming.
- **MESSAGE PROCESSING**
→ Slow consumers → backlog.
- **FAILED MESSAGES**
→ Repeated failures → poison messages.



4. EXTERNAL APIS

- **DEPENDENCY TIMEOUTS**
→ Slow or unreachable APIs.
- **RETRY STORMS**
→ Too many retries make outages worse.
- **CIRCUIT BREAKERS**
→ Prevent cascading failures.



5. DISTINGUISHING FAILURES

INFRASTRUCTURE FAILURE

- High CPU/Memory
- Disk full / slow
- Network latency
- DB down
- DNS issues
- Timeouts at system level

APPLICATION FAILURE

- Bad code
- Memory leaks
- Exceptions
- Logic errors
- Poor queries
- Resource misuse

VS



PRODUCTION NOTE

HEALTHY PODS DO NOT MEAN A HEALTHY SERVICE.



- ✓ CHECK DEPENDENCIES, NOT JUST PODS.
- ✓ MONITOR LATENCY, NOT JUST AVAILABILITY.
- ✓ WATCH QUEUES, DATABASES, AND CACHES.
- ✓ SYSTEMS FAIL TOGETHER, NOT ALONE.



OBSERVE. CORRELATE. ISOLATE. THEN FIX THE REAL ROOT CAUSE.



14. HANDLING PRODUCTION INCIDENTS

Operate methodically when production is already failing.



- 1  Confirm impact
- 2  Establish timeline
- 3  Identify affected services
- 4  Check recent changes
- 5  Stabilize first
- 6  Rollback decisions
- 7  Traffic reduction
- 8  Scaling
- 9  Failover
- 10  Communication
- 11  Evidence preservation
- 12  Incident ownership
- 13  Escalation
- 14  Verification
- 15  Avoiding random changes

KEY PRINCIPLES

- ✓ People first, then systems
- ✓ Stabilize before deep investigation
- ✓ Communicate early and clearly
- ✓ One source of truth
- ✓ Learn and prevent recurrence



PRODUCTION FAILURE

Troubleshooting without coordination can create a second incident.



15. SAFE PRODUCTION TROUBLESHOOTING

Debug without making the outage worse.

- 1  Observe before changing
- 2  Form a hypothesis
- 3  Test one variable
- 4  Record actions
- 5  Compare healthy systems
- 6  Read-only commands first
- 7  Avoid destructive commands
- 8  Preserve logs
- 9  Check dependencies
- 10  Understand blast radius
- 11  Use controlled experiments
- 12  Know rollback paths
- 13  Validate after every change
- 14  Stop when evidence contradicts the hypothesis

TROUBLESHOOTING DO'S



- ✓ Stay calm and think clearly
- ✓ Work step by step
- ✓ Gather evidence
- ✓ Change one thing at a time
- ✓ Keep communication open
- ✓ Document everything
- ✓ Verify the impact
- ✓ Learn from every incident

TROUBLESHOOTING DON'TS



- ✗ Make random changes
- ✗ Skip the observation step
- ✗ Ignore the blast radius
- ✗ Delete logs or data
- ✗ Use destructive commands
- ✗ Assume without evidence
- ✗ Forget to communicate
- ✗ Leave without validation



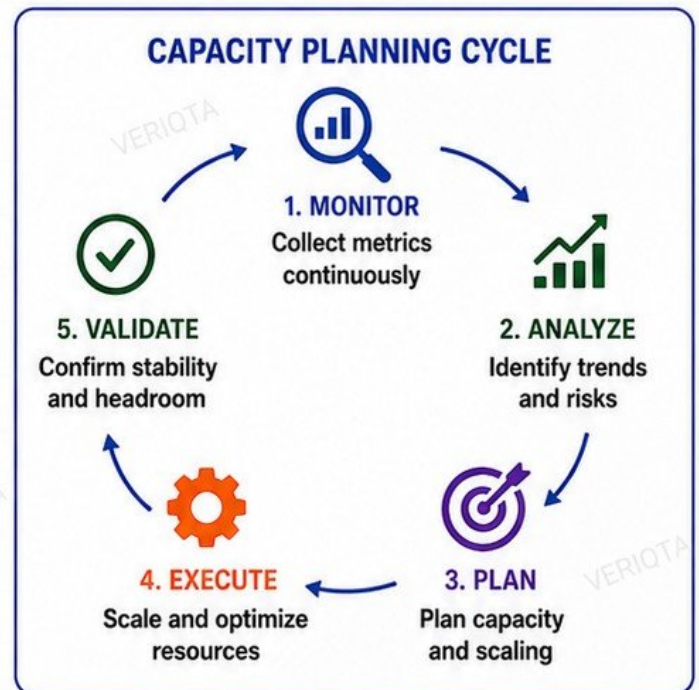
DEBUGGING INSIGHT

Every troubleshooting action should answer a specific question.

16. CAPACITY, SCALING, AND RESOURCE PLANNING

Detect tomorrow's outage in today's metrics.

- 1  Traffic growth
- 2  CPU trends
- 3  Memory trends
- 4  Storage growth
- 5  Database connections
- 6  Queue depth
- 7  Pod scaling
- 8  Node capacity
- 9  HPA behavior
- 10  Cluster Autoscaler behavior
- 11  Auto Scaling Groups
- 12  Resource requests
- 13  Resource limits
- 14  Headroom
- 15  Seasonal traffic
- 16  Forecasting



KEY METRICS TO WATCH

✓ Request rate	✓ Active connections
✓ Error rate	✓ Pod restarts
✓ Latency (P95/P99)	✓ Node pressure
✓ CPU utilization	✓ Autoscaler events
✓ Memory utilization	✓ HPA decisions
✓ Disk usage	✓ Cost trends
✓ Queue depth	✓ Dependencies

SCALING NOTE

Autoscaling cannot compensate for every architectural bottleneck.

17. SECURITY TASKS HIDDEN INSIDE DAILY OPERATIONS

Treat security as operational hygiene.

- 1  Failed authentication
- 2  Suspicious IAM activity
- 3  Privilege changes
- 4  Security group changes
- 5  Public exposure
- 6  Secret leakage
- 7  Vulnerability findings
- 8  Container image vulnerabilities
- 9  Dependency vulnerabilities
- 10  Unusual API activity
- 11  Audit logs
- 12  Kubernetes RBAC
- 13  Privileged workloads
- 14  Image provenance
- 15  Patch status
- 16  Security events

WHAT TO WATCH DAILY



- ✓ Review alerts and notifications
- ✓ Scan recent cloud activity
- ✓ Check access and privilege changes
- ✓ Review security findings
- ✓ Validate configuration changes
- ✓ Monitor exposed resources
- ✓ Review audit and access logs
- ✓ Follow up on open security issues

HIGH RISK INDICATORS



- ✗ Sudden privilege escalation
- ✗ New public-facing resources
- ✗ Unusual geographic logins
- ✗ Large data access or downloads
- ✗ Disabled security controls
- ✗ Outdated or unpatched systems
- ✗ Unknown or unapproved workloads
- ✗ Secrets stored in code or logs

GOOD SECURITY HYGIENE



- ✓ Least privilege access
- ✓ Regular reviews and audits
- ✓ Timely patching
- ✓ Secure configurations
- ✓ Strong monitoring and alerts
- ✓ Quick response and follow-up



SECURITY REMINDER

Operational convenience should never silently become permanent privilege.



18. BACKUPS, RECOVERY, AND DISASTER READINESS

Do not confuse “backup configured” with “data recoverable.”

- 1  Backup job status
- 2  Failed backups
- 3  Snapshot age
- 4  Retention
- 5  Encryption
- 6  Database backups
- 7  Volume snapshots
- 8  Kubernetes persistent data
- 9  Configuration backups
- 10  Terraform state
- 11  Cross-region copies
- 12  Restore testing
- 13  Recovery Point Objective
- 14  Recovery Time Objective
- 15  Disaster recovery readiness
- 16  Evidence of successful restoration

BACKUP BEST PRACTICES

- ✓ Back up what matters
- ✓ Automate and monitor
- ✓ Encrypt all backups
- ✓ Follow the 3-2-1 rule
- ✓ Store offsite or off-region
- ✓ Test restores regularly
- ✓ Document recovery steps
- ✓ Review and improve constantly



THE 3-2-1 RULE



3

3 copies of your data



2

2 different media types



1

1 copy offsite or off-region

WHAT TO VERIFY DAILY / WEEKLY

- ✓ Backup jobs completed successfully
- ✓ No recent failed backups
- ✓ Snapshots are fresh and valid
- ✓ Retention policies are enforced
- ✓ Offsite copies are up to date
- ✓ Restore tests are passing



REAL WORLD LESSON

An untested backup is only a hope.



19. HANDOVER, DOCUMENTATION, AND OPERATIONAL COMMUNICATION

Make sure production knowledge survives shift changes and team boundaries.

- 1  Active incidents
- 2  Degraded systems
- 3  Temporary fixes
- 4  Pending deployments
- 5  Failed pipelines
- 6  Capacity concerns
- 7  Security concerns
- 8  Vendor issues
- 9  Upcoming maintenance
- 10  Expiring certificates
- 11  Open investigations
- 12  Risky changes
- 13  Ownership
- 14  Links to dashboards
- 15  Relevant logs
- 16  Clear next actions

HANDOVER CHECKLIST



- ✓ Update before you hand over
- ✓ Focus on impact, not everything
- ✓ Be specific and actionable
- ✓ Include links, logs, and context
- ✓ Highlight what needs attention
- ✓ State what can wait
- ✓ Answer: Who, What, Where, When, Why
- ✓ Ask questions before signing off
- ✓ Confirm the receiver understands
- ✓ Record in the handover system

GOOD DOCUMENTATION IS

- ✓ Accurate
- ✓ Up to date
- ✓ Easy to find
- ✓ Structured
- ✓ Actionable
- ✓ Consistent
- ✓ Trusted



COMMUNICATION PRINCIPLES

- ✓ Communicate early
- ✓ Communicate clearly
- ✓ Communicate impact
- ✓ Document decisions
- ✓ Avoid assumptions
- ✓ Close the communication loop



ENTERPRISE ADVICE

Good handover prevents engineers from rediscovering the same production context.



VERIQTA
YOUTUBE



VERIQTA
GITHUB



VERIQTA
LINKEDIN



VERIQTA
X



VERIQTA
INSTAGRAM



VERIQTA
TELEGRAM

20. THE DEVOPS ENGINEER'S DAILY OPERATIONS CHECKLIST

Bring the entire operating routine together.

MORNING

-  Production health
-  Overnight alerts
-  Kubernetes
-  Cloud infrastructure
-  Failed pipelines
-  Recent deployments
-  Capacity
-  Backups



DURING THE DAY

-  Monitor production changes
-  Investigate anomalies
-  Review infrastructure plans
-  Support deployments
-  Watch reliability signals
-  Resolve recurring failures
-  Improve automation
-  Document operational discoveries



BEFORE FINISHING

-  Check unresolved alerts
-  Review active incidents
-  Verify production stability
-  Record temporary changes
-  Update tickets and documentation
-  Hand over unresolved risks
-  Confirm scheduled work
-  Leave production in a known state



FINAL PRINCIPLE:

A strong DevOps engineer does not spend every day firefighting.

The goal of daily operations is to detect risk early, understand production continuously, automate repetitive work, make changes safely, and prevent today's small warning from becoming tomorrow's outage.



VERIQTAYOUTUBE



VERIQTAGITHUB



VERIQTALINKEDIN



VERIQTAX



VERIQTAINSTAGRAM



VERIQTATELEGRAM